

Redução Transitiva de Grafos: Um Estudo Comparativo de Desempenho e Complexidade

Alessandra Faria Rodrigues, Daniel Victor Rocha Costa, Felipe Barros Ratton de Almeida, Gabriel Chaves Mendes, Gabriela de Assis dos Reis, Guilherme Henrique da Silva Teodoro, Gustavo Henrique R. de Castro [Pontifícia Universidade Católica de Minas Gerais]

Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais (PUC Minas), Belo Horizonte, MG, 30535-901, Brasil.

Recebido: 28 Junho 2026

Resumo. A redução transitiva de um grafo consiste em remover arestas redundantes sem alterar a atingibilidade entre os vértices. Este trabalho descreve implementações em C++ baseadas em caminhamento no grafo, comparando versões por DFS, BFS, programação dinâmica com *bitsets*, matriz de Warshall e tratamento de componentes fortemente conexos por Tarjan. O relatório justifica a representação adotada, descreve os experimentos realizados, valida os resultados por comparação de atingibilidade e discute os limites da aplicação em grafos não-direcionados. Caminhos Hamiltonianos são mantidos apenas como estudo complementar de caso-limite, pois não constituem, em grafos gerais, uma solução correta para redução transitiva.

Keywords: Redução Transitiva, Atingibilidade, Algoritmos em Grafos, Complexidade, DAG, Componentes Fortemente Conexos

1 Introdução

Dado um grafo direcionado $G = (V, E)$, a atingibilidade de um vértice u é o conjunto de vértices que podem ser alcançados a partir de u . A redução transitiva busca produzir um subgrafo $G' = (V, E')$, com $E' \subseteq E$, tal que a atingibilidade de todo vértice em G' seja igual à atingibilidade do mesmo vértice em G . Assim, uma aresta direta $u \rightarrow v$ só pode ser removida quando já existe outro caminho de u até v que não dependa dessa aresta. Por exemplo, se existem as arestas $A \rightarrow C$, $C \rightarrow B$ e $A \rightarrow B$, a aresta $A \rightarrow B$ é redundante, pois o caminho $A \rightarrow C \rightarrow B$ preserva a mesma relação de alcance.

O objetivo deste trabalho é implementar e comparar algoritmos de redução transitiva baseados em caminhamento, avaliando custo computacional, estrutura de dados e comportamento em diferentes topologias. O foco do relatório é a preservação da atingibilidade, e não a obtenção do menor número absoluto de arestas por qualquer critério estrutural. Essa distinção é importante porque substituir um grafo por uma rota única, como um caminho Hamiltoniano, pode eliminar relações de alcance existentes no grafo original. Por isso, tal ideia é tratada apenas como experimento complementar e não como algoritmo principal.

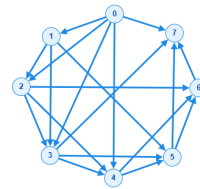
2 Representação do Grafo e Instâncias de Teste

A estrutura principal adotada foi a lista de adjacência, com custo de memória $O(V + E)$. Essa escolha é adequada para algoritmos baseados em caminhamento, pois DFS e BFS percorrem apenas vizinhos existentes, sem varrer posições vazias como ocorreria em uma matriz para grafos esparsos. Além disso, a lista facilita a remoção de arestas redundantes

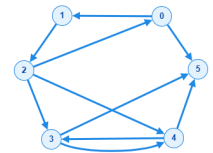
em uma cópia do grafo original, preservando a entrada para novas comparações.

A matriz de adjacência foi utilizada apenas na implementação baseada em Warshall. Nesse caso, o custo de memória $O(V^2)$ é aceitável porque o algoritmo trabalha diretamente com relações booleanas entre todos os pares de vértices. A matriz permite consulta direta a uma possível aresta $u \rightarrow v$, mas é menos econômica quando E é muito menor que V^2 .

Foram usadas instâncias que exercitam situações distintas: um DAG com muitas arestas transitivas, um grafo com ciclos e componentes fortemente conexos, uma instância densa para observar o impacto da representação em memória e uma instância ramificada para verificar se a remoção de atalhos preserva a atingibilidade mesmo quando escolhas locais parecem ambíguas. As Figuras 1 e 2 ilustram esses casos.

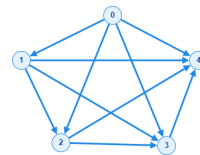


(a) DAG com atalhos.

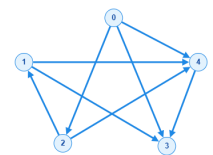


(b) Grafo com ciclos.

Figure 1. Instâncias principais para redução transitiva.



(a) Caso denso.



(b) Caso ramificado.

Figure 2. Instâncias auxiliares para testar preservação de atingibilidade.


```

[LOG TARJAN] Passo 1: Executando Algoritmo de Tarjan para encontrar SCCs...
[LOG TARJAN] Passo 2: Condensando as SCCs encontradas em um DAG...
[LOG TARJAN] Passo 3: Realizando Redução Transitiva no Grafo Condensado...
[LOG TARJAN] Passo 4: Expandindo SCCs em Anéis e montando Sub-grafo
Final: [ANEL SCC] Adicionada aresta de ciclo vital: 4 → 2
        [ANEL SCC] Adicionada aresta de ciclo vital: 2 → 1
        [ANEL SCC] Adicionada aresta de ciclo vital: 1 → 0
        [ANEL SCC] Adicionada aresta de ciclo vital: 0 → 2
        [PONTE SCC] Restaurada aresta essencial entre grupos: 4 → 2
        [PONTE SCC] Restaurada aresta essencial entre grupos: 2 → 4
[LOG TARJAN] Redução concluída com sucesso gerando sub-grafo!

===== Resultado: Tratamento de Ciclos SCC (Tarjan) =====
Vertice 0 aponta para: 2
Vertice 1 aponta para: 0
Vertice 2 aponta para: 1 4
Vertice 3 aponta para: 4
Vertice 4 aponta para: 2 5
Vertice 5 aponta para: (ninguem)
    
```

Figure 6. Identificação de SCCs e redução no DAG condensado.

lapsam componentes inteiras, tornando a condensação por Tarjan e a subsequente redução transitiva estruturalmente inócuas. A única solução formalmente válida para este caso é a extração direta de uma floresta geradora.

4 Critério de Correção da Saída

Para evitar que a remoção de arestas seja confundida com simplificação estrutural arbitrária, a saída foi validada por comparação de atingibilidade. Para cada instância, calcula-se a matriz de alcance do grafo original G e a matriz de alcance do subgrafo reduzido G' . A redução é considerada correta apenas se, para todo par (u, v) , vale $alcance_G[u][v] = alcance_{G'}[u][v]$. Além disso, cada aresta removida deve possuir justificativa local: ao ser retirada, ainda deve existir um caminho alternativo de u até v .

Esse critério separa a implementação correta de heurísticas que apenas diminuam o número de arestas. Uma solução pode produzir um grafo visualmente menor e, ainda assim, estar errada se apagar uma relação de alcance. Essa validação também foi usada para comparar DFS, BFS, Warshall e *bitset*: quando aplicáveis à mesma instância, as abordagens devem gerar subgrafos equivalentes quanto à atingibilidade, ainda que a ordem de processamento ou a representação interna sejam diferentes.

4.1 Protocolo Experimental

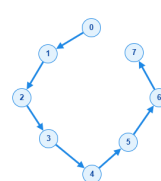
Em cada execução foram registrados: número de vértices, número de arestas originais, número de arestas mantidas após a redução, abordagem usada e resultado da comparação de atingibilidade. O número de arestas removidas, isoladamente, não foi tratado como prova de qualidade, pois uma remoção agressiva pode gerar um subgrafo menor e incorreto. Por isso, a métrica principal foi booleana: a matriz de atingibilidade de G' deve ser idêntica à matriz de atingibilidade de G .

O tempo de execução foi usado apenas para comparar abordagens corretas. Essa decisão evita premiar heurísticas rápidas que não preservam alcance. Nos DAGs, a comparação enfatizou o custo de recomputar caminhamentos em DFS/BFS contra o reaproveitamento de estados no método com *bitset*. Nos grafos cíclicos, a análise destacou a necessidade de decompor o grafo em SCCs antes de aplicar raciocínio de DAG. Desse modo, os experimentos ficam alinhados ao enunciado: primeiro preservar atingibilidade; depois comparar desempenho.

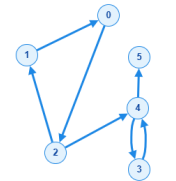
5 Experimentos e Resultados

Nos DAGs, DFS e BFS produziram a mesma redução, pois ambas testam a existência de caminhos alternativos antes de remover uma aresta. A diferença observada está no custo: como cada aresta pode disparar uma nova busca, a repetição de caminhamentos torna essas versões menos eficientes quando o grafo possui muitos atalhos transitivos. A versão com *bitset* reduziu esse custo prático ao reaproveitar conjuntos de atingibilidade já calculados.

No grafo cíclico, a etapa de Tarjan foi usada para separar o problema em duas partes: preservação interna das SCCs e redução das arestas entre componentes. Esse tratamento é mais correto do que aplicar ordenação topológica diretamente sobre o grafo original, pois a ordenação topológica só existe em DAGs. As Figuras 7 mostram os subgrafos obtidos nas instâncias principais.



(a) Resultado em DAG.



(b) Resultado com SCCs.

Figure 7. Subgrafos após remoção de arestas transitivas.

Table 1. Comparação das abordagens implementadas.

Abordagem	Tempo	Memória	Observação
DFS/BFS por aresta	$O(E(V + E))$	$O(V + E)$	Simples, mas recalcula muitos caminhamentos.
Bitset em DAG	$O((V + E)V/w)$	$O(V^2/w)$	Reaproveita atingibilidade com operações por palavra.
Warshall	$O(V^3 + EV)$	$O(V^2)$	Direto e robusto, porém custoso em grafos grandes.
Tarjan + DAG condensado	$O(V + E) + \text{redução do DAG}$	$O(V + E)$	Trata ciclos sem assumir anel interno inexistente.

Table 2. Comparação das abordagens implementadas e tempos obtidos nos testes.

Abordagem	Tempo teórico	Tempo medido	Memória	Observação
DFS por aresta	$O(E(V + E))$	2,648s	$O(V + E)$	Simples e aderente ao caminhamento, mas recalcula muitas buscas.
BFS por aresta	$O(E(V + E))$	4,936s	$O(V + E)$	Também preserva atingibilidade, porém teve maior custo experimental.
Bitset em DAG	$O((V + E)V/w)$	3,785s	$O(V^2/w)$	Reaproveita conjuntos de atingibilidade com operações por palavra de máquina.
Warshall	$O(V^3 + EV)$	1,778s	$O(V^2)$	Foi rápido nas instâncias pequenas, mas possui custo cúbico em grafos maiores.
Tarjan + DAG condensado	$O(V + E) + \text{redução do DAG}$	1,267s	$O(V + E)$	Identifica SCCs e evita tratar ciclos como erro; não assume anel interno inexistente.
Guloso	$O(V + E)$	1,543s	$O(V + E)$	Experimento complementar; pode achar rota simples, mas não garante redução transitiva geral.
Topológico	$O(V + E)$	1,460s	$O(V + E)$	Útil em DAGs para verificar caso-limite de cadeia, não substitui a redução por atingibilidade.
Torneio	$O(V^2)$	1,236s	$O(V + E)$	Caso especial baseado em torneios; não generaliza para grafos direcionados arbitrários.

6 Estudo Complementar: Caminhos Hamiltonianos

Os experimentos com Caminhos Hamiltonianos foram posicionados como análise complementar de caso-limite. Investigou-se não se eles resolvem a redução em grafos

gerais, mas quando uma rota única coincidiria com uma redução válida. Isso ocorre estritamente quando a rota preserva a exata atingibilidade do grafo original.

Em DAGs, uma ordenação topológica única indica um caminho Hamiltoniano onde a cadeia coincide com a redução, comportando-se como ordem total. Sem essa propriedade, impor uma rota única apaga vértices alcançáveis em ramificações laterais. Em grafos de torneio, o Teorema de Rédei garante o caminho, mas não a redução transitiva, pois instâncias cíclicas possuem relações bidirecionais que se perdem ao isolar apenas uma cadeia.

A heurística gulosa por maior grau de saída foi tratada como experimento negativo. Embora encontre rotas visualmente compactas, não garante a preservação da atingibilidade, sendo descartada como solução principal. O valor desta seção é delimitar um erro conceitual claro: reduzir arestas difere de substituir o grafo por um percurso.

Na implementação, os módulos Hamiltonianos permaneceram apenas como ferramentas auxiliares. O caso de torneio provou que existir um caminho não implica manter as relações de alcance, e o método guloso evidenciou que escolhas locais não substituem a verificação formal de atingibilidade. Assim, aproveita-se o trabalho implementado sem comprometer a integridade conceitual da solução.

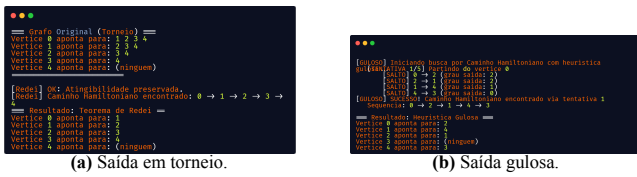


Figure 8. Execuições Hamiltonianas mantidas como evidência experimental complementar.



Figure 9. Experimentos Hamiltonianos usados apenas como casos-limite, não como redução transitiva geral.

Grafos não-direcionados. Em grafos não-direcionados, Caminhos Hamiltonianos também não são solução geral: podem preservar conectividade se existirem, mas são desnecessários e difíceis de encontrar. Para preservar atingibilidade nesse caso, basta construir uma floresta geradora por DFS ou BFS.

7 Conclusão

O trabalho implementou e comparou abordagens de redução transitiva baseadas em caminhamento no grafo. A lista de adjacência foi a representação mais adequada para DFS, BFS, *bitset* em DAG e Tarjan, enquanto a matriz se justificou no método de Warshall. A análise mostra que DFS e BFS são didáticas e corretas, mas caras no pior caso; *bitsets* melhoraram o desempenho prático em DAGs ao compactar a atingibilidade; Warshall oferece uma solução matricial simples; e Tarjan permite tratar grafos cíclicos por meio de SCCs e do DAG condensado.

A principal correção conceitual é que Caminhos Hamiltonianos não constituem, em geral, redução transitiva. Eles só preservam atingibilidade em casos particulares, quando a relação de alcance do grafo original já é compatível com uma cadeia. Por isso, a solução entregue deve ser avaliada pela preservação rigorosa da atingibilidade, conforme a definição do enunciado, e não pela obtenção de uma rota única ou estruturalmente mais simples.

Declarações

Authors' Contributions

A execução do projeto foi estruturada da seguinte forma: Alessandra e Gabriel ficaram responsáveis pela implementação das versões por DFS, BFS e programação dinâmica em DAGs; Daniel e Gabriela desenvolveram os testes, a coleta de resultados e a organização das instâncias; Guilherme conduziu a análise de complexidade e a documentação técnica; Felipe e Gustavo revisaram o tratamento de grafos cíclicos, a discussão sobre grafos não-direcionados e os experimentos complementares com Caminhos Hamiltonianos como casos-limite. Todos os membros participaram da revisão final, executaram os experimentos e devem ser capazes de explicar a solução completa.

Competing interests

Os autores declaram que não possuem conflitos de interesse.

Availability of data and materials

Os códigos-fonte desenvolvidos em C++ encontram-se disponíveis para consulta e replicação dos experimentos no repositório oficial do GitHub: https://github.com/GabrielChavesMendes/Trabalho_Projeto_Analise_Algoritmo.

References

- [1] ALMEIDA, C. S. J.; PATROCINIO JR, Zenilton Kleber G. do; MALINOWSKI, S.; GRAVIER, G.; GUIMARAES, Silvio Jamil Ferzoli. Decreasing graph complexity with transitive reduction to improve temporal graph classification. *International Journal of Data Science and Analytics*, v. 19, p. 229–242, 2025.
- [2] AHO, A. V.; GAREY, M. R.; ULLMAN, J. D. The transitive reduction of a directed graph. *SIAM Journal on Computing*, v. 1, n. 2, p. 131–137, 1972.
- [3] TARJAN, R. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, v. 1, n. 2, p. 146–160, 1972.
- [4] WARSHALL, S. A theorem on boolean matrices. *Journal of the ACM*, v. 9, n. 1, p. 11–12, 1962.
- [5] RÉDEI, L. Ein kombinatorischer Satz. *Acta Litterarum ac Scientiarum*, v. 7, p. 39–43, 1934.
- [6] TANG, X. *et al.* One edge at a time: a novel approach towards efficient transitive reduction computation on DAGs. *IEEE Access*, v. 8, p. 38010–38022, 2020.